A Hybrid-Parallel Decision Tree: MPI, OpenMP and CUDA Scaling of Histogram-Based CART

[Member 1 Name (A0000000X)] [Member 2 Name (A0000000X)]
[Member 3 Name (A0000000X)]

[**TODO:** public GitHub repository link]

1 Introduction

A decision tree is built greedily: starting from all the training rows at the root, each node chooses the one feature and threshold that best separates the two classes, splits its rows accordingly, and repeats on the two children until a stopping rule fires. This project parallelizes the *construction* of a binary classification tree and studies how different parallelization choices affect its scaling and efficiency.

The problem is a good parallelization target because it is *not* embarrassingly parallel. Although the training rows can be partitioned across processes trivially, choosing a split needs the class counts on both sides of every candidate threshold *summed over all rows*. With the rows spread across processes, no process can decide a split alone: each node forces a collective communication, and the children a node produces determine the work of the next level. That per-node global reduction, and the dependency between levels, are the source of the communication and load-balancing challenges we examine.

The report follows the project from the algorithm and its serial cost model (Sections 2–3), through a correctness validation (Section 4), to the three scaling studies: serial problem-size scaling (Section 5), pure-MPI strong and weak scaling to 256 cores (Section 6), and the MPI+X results for the MPI+OpenMP hybrid and the CUDA GPU arm (Section 7). Reproduction instructions are in `README.md`.

2 The algorithm and the computing environment

2.1 Histogram-based CART with the Gini criterion

For a node holding t samples of which p are positive, the Gini impurity is

$$G(p,t) = 1 - \left(\tfrac{p}{t}\right)^2 - \left(\tfrac{t-p}{t}\right)^2, \tag{1}$$

and a candidate split into a left child (p_L, t_L) and right child (p_R, t_R) is scored by the Gini gain

$$\Delta G = G(p,t) - \frac{t_L}{t}\, G(p_L, t_L) - \frac{t_R}{t}\, G(p_R, t_R). \tag{2}$$

The node adopts the (feature, threshold) with the largest ΔG, and recursion stops when the node reaches the maximum depth, holds at most `min-split` samples, is pure, or no split gives positive gain.

Rather than sorting each feature at every node, we use a **histogram** formulation. Each feature's values are placed into bins; when a feature has at most 256 distinct values the bins are its exact distinct values, so the candidate thresholds are identical to a sort-based search. A node is then summarised by, per bin, the pair [count, positive count]. Scanning those bins in order reproduces every candidate split from Equation 2. The histogram is the unit of work that the parallelization distributes, and because its entries are integers, summing them across processes is exact and order-independent — a property we rely on for both correctness and determinism (Sections 3–4).

Per node, the dominant cost is building the histogram, which is $O(n_{\text{node}} \times F)$ for n_{node} rows and F features. Summed over the tree, training cost grows with both the number of rows N and the number of features F; Section 5 measures both.

2.2 One source, four configurations

The algorithm is written once in `src/dt.cpp`; MPI and OpenMP are enabled by compile-time guards so the split-search logic is byte-identical in every build. This yields the four configurations the report compares:

- **serial** — no MPI, no OpenMP; the single-core baseline for speedup;
- **pure MPI** — the MPI build run with `OMP_NUM_THREADS=1`, so MPI is the only parallelism (Section 6);
- **MPI+OpenMP** — the same build with multiple threads per rank (Section 7);
- **CUDA** — a separate GPU implementation of the histogram hot path (Section 7).

2.3 Computing environment

Benchmarks were run on [**TODO:** Atlas / Vanda — confirm]. The run recorded in `results/raw/env_*.txt` used a dual-socket **AMD EPYC 7R13** node (48 cores, one thread per core), **OpenMPI 4.1.4**, and **GCC 12.2.0** (module `foss/2022b`); each benchmark point is the mean of several repetitions after a warm-up tree. [**TODO:** replace with the final 256-core run's environment once collected.]

2.4 Dataset

We use the preprocessed Instacart reorder dataset `instacart_clean.csv`: 200 000 orders, 7 features (order number, day of week, hour of day, days since prior order, add-to-cart order, aisle id, department id) and a binary `reordered` label. The file is clean (no missing values) and every feature has at most 256 distinct values, so binning is exact. [**TODO:** confirm whether this file is the real Kaggle extract or the preprocess.py schema-matched sample, and state it plainly.]

Because 200 000 rows are far too few to occupy hundreds of cores — the per-node reduction would dominate — the scaling studies instead use a deterministic synthetic generator that reproduces the Instacart schema and value ranges. Row r is a pure function of (seed, r), so the dataset is identical for every process count and nothing is read from disk, letting us vary N and F freely. The synthetic data is Instacart-*like* in shape, not sampled from the real file; real-data conclusions (Section 8) come from the real file itself.

3 Parallelization design

Data parallelism over rows, not over tree nodes. The rows are block-distributed across processes once, at load time, and never move again. For each node, every process builds the histogram of *its own* rows and a single `MPI_Allreduce` with integer sum produces the global histogram; every process then scans that identical histogram and independently reaches the identical split, so no split decision needs to be broadcast.

We deliberately did not parallelize by handing whole subtrees to different processes (task parallelism). Near the root there are only a handful of nodes but the most data, so subtree parallelism is scarcest exactly where the cost is greatest; sibling subtrees are unbalanced, so processes would idle; and assigning a subtree to a process would require reshuffling rows at every split. Data parallelism instead keeps every process busy on every node and attacks the cost where it lives — the histogram build, which Section 6 shows dominates the runtime.

Determinism as a design constraint. Because the reduction sums integers, its result does not depend on the number of processes or the order of summation, so the tree is bit-for-bit identical across all configurations. This both gives a strong correctness check (Section 4) and ruled out alternatives that would have broken it, such as reducing floating-point partial gains or assigning subtrees to processes.

Two communication patterns (tree growth). The same tree can be grown two ways, trading off message count against message size. *Node-wise* growth is depth-first with one reduction per internal node (many small messages, latency-bound); *level-wise* growth is breadth-first and reduces the stacked histograms of all nodes at one depth in a single collective (far fewer, larger messages). At depth 6 this is roughly 64 reductions versus 7. [**TODO:** report the measured counts and the runtime difference from `results/raw/design_*.csv`, and confirm level-wise is faster.]

Two OpenMP strategies (X = OpenMP). Within a rank, threads can either split the *rows* (each thread fills a private histogram, then a parallel reduction combines them) or split the *features* (threads share one histogram, with parallelism capped by F). [**TODO:** report which wins from `design_*.csv`; the row split is expected to scale better as it is not limited by the feature count.]

4 Validating correctness

Two independent checks establish that the parallel builds compute the right tree.

First, **cross-configuration equivalence** exploits the determinism above. The script `scripts/check_equivalenc` hashes the entire tree (structure, split features, thresholds and leaf predictions) with FNV-1a and requires every configuration — process counts $\{1, 2, 3, 4, 7\}$, thread counts $\{1, 2, 3\}$, both growth modes, both OpenMP strategies, both the synthetic and CSV input paths, on a prime number of rows so the row blocks are uneven — to reproduce the serial reference hash exactly. [**TODO:** paste the final "N / N configurations identical to the reference" line from a run.]

Second, an **external reference**: `scripts/compare_sklearn.py` rebuilds the tree dumped by `dt --dump-tree` and compares it node by node against scikit-learn's `DecisionTreeClassifier` on the same data (the only permitted differences are exact ties between equally good splits, which the two implementations break differently). [**TODO:** run on the Instacart CSV and report the agreement.]

This equivalence was also what motivated the current design. An earlier MPI implementation combined rank-local split gains with `MPI_MAXLOC`, which is not the global Gini gain; its trees drifted with the process count (training accuracy 68.46 % at one rank versus 68.42 % at eight on the same 200 000 rows), while the root split stayed correct and so hid the bug from shallow checks. The histogram-reduction design removes this drift. [**TODO:** confirm/restate these dev-time numbers or drop if not reproducible.]

5 Serial scaling with problem size

The serial build establishes both the speedup baseline and the cost model. We vary the two dimensions of problem size independently: the number of rows N at fixed F, and the number of features F at fixed N. Because per-node work is $O(n_{\text{node}} \times F)$, both should give near-linear runtime growth.

[**TODO:** run the serial sweep (ranks=threads=1): $N \in \{1, 2, 4, 8, 16, 32\}$M at fixed F, and $F \in \{7, 14, 28, 56, 112, 224\}$ at fixed N. Source: `results/raw/serial_scan_*.csv`.]

[**TODO:** figure: serial runtime vs N at fixed F (expect linear).] [**TODO:** figure: serial runtime vs F at fixed N (expect linear).] [**TODO:** one paragraph: confirm the $O(N \times F)$ model from the two slopes.]

6 Pure-MPI strong and weak scaling

This section uses the pure-MPI configuration (one rank per core, `OMP_NUM_THREADS=1`). Speedup is $S_p = T_1/T_p$ against the serial baseline on the same data, and parallel efficiency is $E_p = S_p/p$ for p cores; T_p is the maximum over ranks of the barrier-to-barrier training time.

6.1 Strong scaling

Here the problem is fixed and the core count grows, up to 256 cores.

[**TODO:** table + figure: T_p, S_p and E_p vs core count to 256. Source: `results/raw/strong_*.csv` and `strong_scaling.*`. Discuss where efficiency starts to fall and why.]

6.2 Weak scaling

Here the work per core is fixed (constant rows per rank) and both N and the core count grow together, so ideal efficiency is flat at 1.

[**TODO:** table + figure: E_p vs core count at constant rows/core to 256. Source: `results/raw/weak_*.csv` and `weak_scaling.*`.]

6.3 The impact of communication

With `--profile`, a barrier before each reduction separates time spent *waiting* for the slowest rank (load imbalance) from time spent in the reduction itself (transfer). Decomposing the runtime into histogram build, split scan, partition, wait and reduction shows how the parallel part is spent and why efficiency falls at high core counts.

[**TODO:** figure: stacked runtime breakdown (t_{hist}, t_{scan}, t_{part}, t_{wait}, $t_{\text{allreduce}}$) vs cores from the profiled runs (`*_breakdown.*`). Discuss: histogram build dominates and shrinks with p, while wait and reduction grow, eventually capping speedup.]

7 MPI+X: OpenMP hybrid and the GPU

7.1 MPI + OpenMP

Adding OpenMP threads within each rank reduces the number of ranks sharing the reduction while keeping the cores busy. At a fixed total core count we vary the ranks×threads split to find the best balance between MPI reduction cost and on-node thread scaling.

[**TODO:** table + figure: runtime at fixed cores for splits such as 48×1, 24×2, 12×4, 6×8 from `results/raw/hybrid_*.csv` and `hybrid_fixed.*`; state the best ratio and explain why fewer ranks reduce communication but too many threads lose on-node efficiency.]

7.2 CUDA on a GPU

Because the chosen X is OpenMP, we additionally implement the histogram hot path on a GPU in CUDA and benchmark it on at least one GPU. The kernel builds the same [count, positive count] histogram the CPU builds, with threads accumulating over rows into per-bin counters, so the split chosen matches the CPU result.

[**TODO:** CUDA implementation is being rewritten (the previous kernel tested hard-coded thresholds rather than the data's bins and evaluated only the root). Once complete: report the GPU model, the data size, the histogram/split runtime on the GPU versus one CPU core and versus a full OpenMP node, and the host↔device transfer cost. Source: GPU benchmark run.]

8 Discussion and conclusions

How the problem was parallelized. [**TODO:** summarise, tying the sections together: data-parallel over rows with a per-node integer-histogram `Allreduce`; the reduction is the only communication and therefore the main scaling limit; level-wise growth minimises the number of collectives; OpenMP adds on-node parallelism over rows and sibling nodes; CUDA accelerates the same histogram on a GPU. State where efficiency is lost (reduction latency and load imbalance at high core counts) using the Section 6 breakdown.]

Conclusions about the data. On the real Instacart file the serial tree reaches [**TODO:** confirm] about 68.5 % training accuracy at depth 6, against a majority-class baseline of 66.2 % (33.8 % of orders are reorders), a modest but real improvement. [**TODO:** name the top split features from `--dump-tree` and interpret them (e.g. add-to-cart order and days since prior order), and note that 200 000 rows is far too small to need 256 cores — which is why scaling uses synthetic data.]

9 AI usage

[**TODO:** Write an honest declaration. A truthful template to edit: "Anthropic's Claude, via the Claude Code CLI, assisted with code review, with diagnosing the MPI split-search correctness issue, and with drafting and structuring this report. All design decisions, the choice of algorithm and

parallelization strategy, and the final analysis were the authors'. All AI-assisted code was reviewed, compiled and run by the authors, and every reported number comes from an actual run on [**TODO:** Atlas/Vanda]."]